

Predicción de Abandono de Clientes en Telecomunicaciones

Pipeline ML + SHAP + LLM (Groq) para detección y explicación de churn

Kadiha Muhamad Orta Laura Restrepo Berrio Johan Rico Nivia
knmuhamado@eafit.edu.co lrestrepb1@eafit.edu.co jsricon@eafit.edu.co

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT
Curso: Inteligencia Artificial . Semestre 2026-1
Entrega: 19–22 de mayo de 2026

Repositorio: <https://github.com/Johan-Rico/Churn-Prediction-LLM>

Resumen

Contexto: El abandono de clientes (churn) es uno de los problemas más costosos en la industria de telecomunicaciones; retener un cliente existente es significativamente más económico que adquirir uno nuevo.

Objetivo: Construir un pipeline reproducible que prediga qué clientes abandonarán el servicio y explique las razones en lenguaje natural.

Método: Exploración del dataset Telco Customer Churn (Kaggle, ≈ 7.000 registros); baseline con Regresión Logística; modelos alternativos con Random Forest y XGBoost; interpretabilidad global y local con SHAP; generación automática de reportes en español mediante un LLM (Groq).

Resultados: Baseline (LR): Accuracy = 0.72, AUC-ROC = 0.8414, F1 = 0.6164. Random Forest: AUC-ROC = 0.8430. Modelo XGBoost: Accuracy = 0.7516, AUC-ROC = 0.8393, F1 = 0.6285.

Conclusión: El sistema combina predicción precisa con explicaciones accionables en lenguaje natural, facilitando decisiones de retención para usuarios no técnicos.

Palabras clave: Churn prediction, XGBoost, SHAP, LLM, Groq, Streamlit, Telco.

1. Introducción

El abandono de clientes (churn) representa una pérdida directa de ingresos para empresas de telecomunicaciones. Identificar de forma anticipada qué clientes están en riesgo permite implementar estrategias de retención focalizadas y rentables.

Este proyecto aborda la siguiente pregunta de investigación: *¿Puede un modelo de aprendizaje automático predecir el churn de clientes de telecomunicaciones de forma competitiva, y puede un LLM traducir esa predicción en una explicación accionable en español?*

Se utiliza el dataset *Telco Customer Churn* de IBM/Kaggle [1]. El documento se organiza así: Sección 2 describe los datos y el EDA; Sección 3 la arquitectura del sistema;

Sección 4 la metodología experimental; Sección 5 los resultados; Sección 6 la discusión y Sección 7 las conclusiones.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

Cuadro 1: Resumen del dataset Telco Customer Churn

Atributo	Descripción
Fuente	Kaggle / IBM Sample Data [1]
Registros (N)	7.043 filas
Features (p)	19 numéricas/categóricas
Variable objetivo	Churn (binaria: Yes/No)
Período	Versión 2018
Tamaño	≈ 1 MB

2.2. Hallazgos del análisis exploratorio

El análisis exploratorio realizado en `01_eda.ipynb` reveló los siguientes patrones principales:

- **Desbalance de clases:** La clase positiva (churn) representa el $\approx 26.5\%$ del dataset, lo que implica un desbalance moderado que se maneja con `class_weight='balanced'` en el baseline y `scale_pos_weight` en XGBoost.
- **Tipo de contrato:** Los contratos mes a mes concentran la mayor tasa de abandono ($\approx 42.7\%$). Los contratos anuales o bianuales reducen esa tasa por debajo del 10% , convirtiéndose en el predictor categórico más fuerte.
- **Tenure:** Los clientes que abandonan llevan significativamente menos meses en la empresa. Es la variable numérica con mayor poder discriminante; clientes con menos de 6 meses tienen riesgo notablemente superior al promedio.
- **Cargos mensuales:** Los clientes que abandonan pagan más mensualmente. Esta variable está correlacionada con el uso de fibra óptica, lo que genera cierta colinealidad a considerar.

- **Valores nulos:** 11 filas tenían TotalCharges nulo porque su tenure era 0; se imputaron a 0 al no tener historial de facturación.
- **Variables más relevantes:** El EDA identificó tenure, Contract, MonthlyCharges, InternetService, TechSupport, PaymentMethod y PaperlessBilling como los predictores con mayor asociación con churn, resultado posteriormente confirmado por el análisis SHAP.

Figura 1 — Desbalance de clases en el dataset

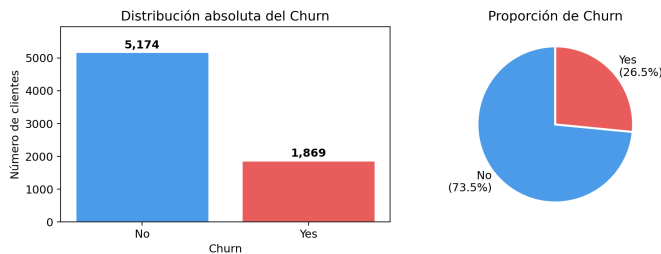
Figura 1: Distribución de la variable objetivo (Churn). La clase positiva representa el $\approx 26.5\%$ del dataset.

Figura 4 — Correlación entre variables numéricas y Churn

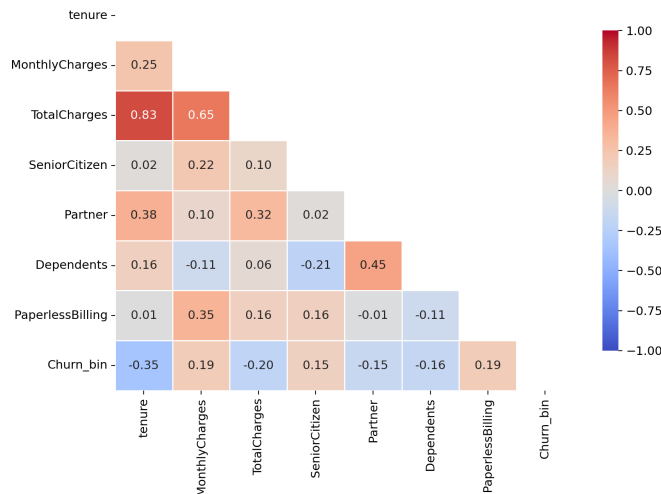


Figura 2: Mapa de correlación entre variables numéricas. Se observa alta correlación entre MonthlyCharges y TotalCharges.

3. Arquitectura del Sistema

El sistema se compone de tres capas que interactúan de forma secuencial (Figura 3): preprocesamiento, predicción con ML y generación de explicaciones en lenguaje natural.

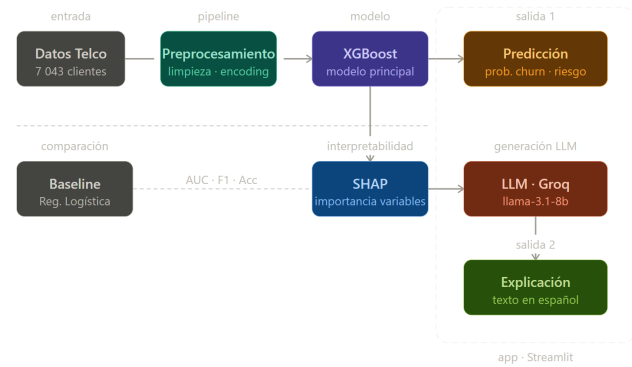


Figura 3: Arquitectura general del sistema propuesto.

3.1. Preprocesamiento (src/preprocessing.py)

La entrada del sistema es un diccionario con los atributos del cliente, tal como los captura el formulario de Streamlit. La función `build_customer_frame` convierte ese diccionario en un DataFrame de una fila y aplica tres transformaciones: (1) elimina `customerID` si está presente, (2) convierte `TotalCharges` a numérico imputando NaN con 0 (los 11 casos del dataset corresponden a clientes con `tenure=0`), y (3) fuerza `SeniorCitizen` a entero.

Posteriormente, `encode_customer_frame` aplica one-hot encoding con `drop_first=True` y realinea las columnas resultantes con `feature_columns.pkl`, el artefacto que preserva el orden exacto de columnas visto durante el entrenamiento. Las columnas ausentes se rellenan con 0 y las columnas extra se descartan. Este mecanismo garantiza que cualquier combinación de entradas del formulario sea compatible con el modelo sin lanzar excepciones en producción.

La división train/test se realizó con proporción 80/20, `random_state=42` y `stratify=y` para preservar la proporción de clases en ambos conjuntos.

3.2. Modelo de ML (src/model.py)

El módulo `model.py` expone la función `predict_churn_with_probability`, que carga `xgb_churn_model.pkl` y `feature_columns.pkl` desde el directorio `models/`, prepara la entrada mediante `prepare_customer_input` y retorna un diccionario con tres campos: `prediction` (0 o 1), `probability` (flotante entre 0 y 1) y `risk_label` (ALTO, MEDIO o BAJO según umbrales de 0.7 y 0.4).

Se entrenaron y compararon tres clasificadores: Regresión Logística (baseline), Random Forest y XGBoost. El modelo final desplegado es XGBoost con hiperparámetros optimizados mediante `GridSearchCV`, persistido en `models/xgb_churn_model.pkl`.

3.3. Componente LLM (src/llm.py)

generar_explicacion_churn recibe el diccionario del cliente y la probabilidad predicha, y construye un prompt estructurado en español que incluye: el nivel de riesgo calculado, los nueve atributos más relevantes identificados en el EDA (tenure, Contract, MonthlyCharges, InternetService, TechSupport, PaperlessBilling, PaymentMethod, Partner, Dependents) y tres instrucciones explícitas: explicar el nivel de riesgo, identificar los factores determinantes y formular una recomendación de retención.

La llamada a la API de Groq [4] usa el modelo llama-3.1-8b-instant con temperature=0.4 y max_tokens=300. La temperatura baja favorece respuestas consistentes y reproducibles entre ejecuciones. Si GROQ_API_KEY no está configurada, la función lanza un ValueError descriptivo que la interfaz captura y muestra al usuario sin interrumpir la predicción numérica.

3.4. Interfaz Streamlit (app/main.py)

La interfaz expone 19 controles en la barra lateral (selectboxes, sliders y un number_input) que cubren todos los atributos del dataset. Al cargar la aplicación se muestran tres métricas globales del dataset mediante @st.cache_data: número de clientes analizados, tasa de churn y tenure promedio. Al presionar Predecir churn, la app invoca secuencialmente predict_churn_with_probability y generar_explicacion_churn, presentando los resultados en dos bloques: métricas numéricas (clase, probabilidad y nivel de riesgo) y el párrafo explicativo del LLM. Un expander opcional muestra el JSON completo enviado al modelo.

4. Metodología

4.1. Configuración experimental

Cuadro 2: Parámetros clave del sistema

Parámetro	Valor
Modelos evaluados	LR (baseline), Random Forest, XGBoost
Modelo desplegado	XGBClassifier
Semilla	SEED = 42
Split	Train/Test 80/20 (estratificado)
Métrica principal	AUC-ROC, F1-score
Desbalance	class_weight='balanced' (LR); scale_pos_weight (XGBoost)
LLM	llama-3.1-8b-instant vía Groq
Prompting	Zero-shot en español
Framework	scikit-learn, xgboost, shap, groq
Hardware	CPU / Google Colab (GPU T4)

4.2. Estrategia de validación

Se utilizó hold-out con split estratificado para preservar la proporción de clases en ambos conjuntos. Las métricas se reportan únicamente sobre el conjunto de test, que no participó en el entrenamiento ni en la selección de hiperparámetros.

4.3. Experimentos realizados

- Baseline:** Regresión Logística con class_weight='balanced'. Sirve como punto de referencia interpretable.
- Random Forest:** Modelo de ensamble evaluado como alternativa intermedia entre el baseline lineal y XGBoost.
- XGBoost:** Modelo principal con hiperparámetros optimizados mediante GridSearchCV (scoring='roc_auc', cv=3).
- SHAP + LLM:** Análisis de importancia de variables con SHAP y generación de explicaciones automáticas en español con Groq.

5. Resultados

5.1. Comparación de modelos

Se evaluaron tres clasificadores sobre el mismo conjunto de test (20 % del dataset, estratificado). La Tabla 3 resume las métricas obtenidas.

Cuadro 3: Comparación de modelos en test set

Modelo	Acc.	F1	AUC
Baseline (LR)	0.72	0.6164	0.8414
Random Forest	—	—	0.8430
XGBoost	0.7516	0.6285	0.8393

Nota: Random Forest alcanza el AUC más alto (0.843), pero XGBoost se seleccionó como modelo de producción por su mayor F1 y accuracy, métricas más relevantes para identificar correctamente la clase positiva en retención de clientes.

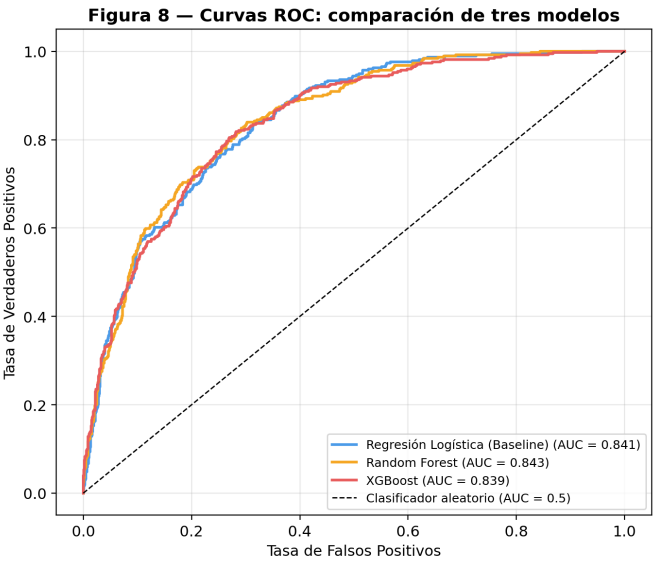


Figura 5: Curvas ROC comparando los tres modelos evaluados. Random Forest (AUC=0.843) supera levemente a LR (0.841) y XGBoost (0.839).

La Figura 4 muestra que tanto la Regresión Logística como XGBoost tienen un rendimiento muy similar en el espacio ROC. La Figura 5 añade Random Forest, que obtiene el AUC más alto (0.843), aunque la diferencia entre los tres modelos es inferior a 0.005 puntos, lo que indica que la dificultad del problema está en la naturaleza del dataset y no en la elección del clasificador.

5.2. Curvas ROC

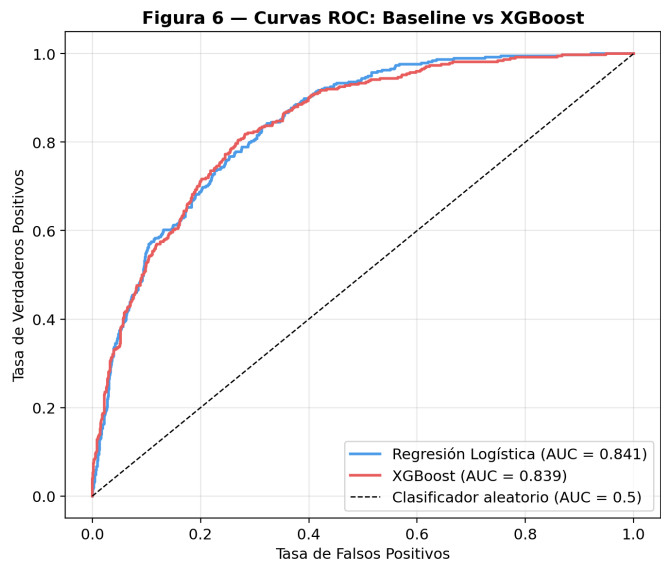


Figura 4: Curvas ROC: Baseline vs. XGBoost. Ambos modelos superan ampliamente al clasificador aleatorio (AUC = 0.5).

5.3. Análisis de importancia (SHAP)

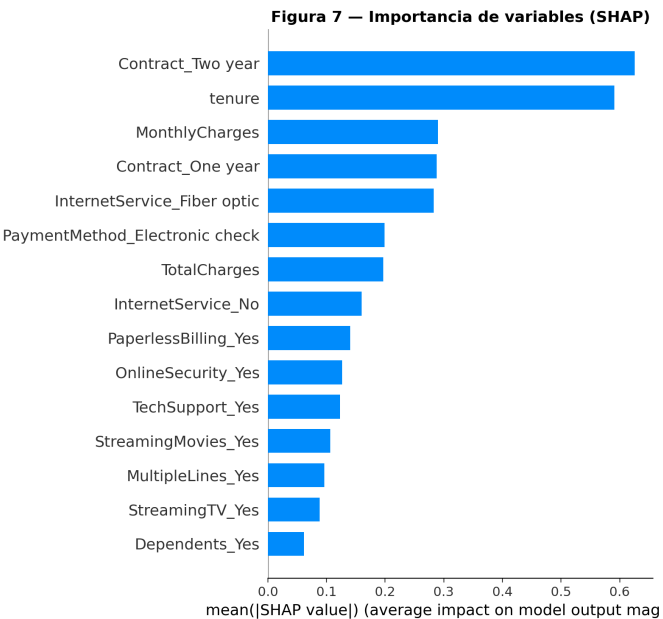


Figura 6: Importancia global de variables (SHAP bar plot).

Figura 7: SHAP beeswarm: dirección e impacto de cada variable.

El análisis SHAP confirma que *Contract*, *tenure* y *MonthlyCharges* son las variables de mayor impacto global, en línea con los hallazgos del EDA. Contratos mes a mes y baja antigüedad empujan la predicción hacia churn, mientras que contratos anuales y mayor *tenure* la empujan hacia retención. *InternetService* (Fiber optic) también contribuye positivamente al riesgo, asociado a cargos mensuales más altos.

5.4. Análisis cualitativo — Ejemplos del LLM

```

1 # Input al modelo
2 cliente = {
3     "tenure": 2,
4     "Contract": "Month-to-month",
5     "MonthlyCharges": 89.5,
6     "InternetService": "Fiber optic",
7     "TechSupport": "No",
8     "PaymentMethod": "Electronic check"
9 }
10 # Probabilidad predicha: 0.82
11
12 # Explicacion generada por Groq (en español):
13 "Este cliente tiene una probabilidad de abandono
14 del 82%. Los principales factores de riesgo son
15 su contrato mensual sin compromiso de permanencia,
16 una antigüedad de solo 2 meses y una factura
17 mensual elevada de $89.50. La combinacion de
18 fibra optica sin soporte tecnico y pago por
19 cheque electronico incrementa adicionalmente
20 el riesgo. Se recomienda ofrecer un descuento
21 por contrato anual con soporte tecnico incluido."
```

Listing 1: Ejemplo de output del sistema LLM para un cliente de alto riesgo

6. Discusión

6.1. Interpretación de resultados

Los tres modelos evaluados convergen en un rango AUC de 0.839–0.843, lo que indica que la mayor parte de la información discriminante está capturada ya por el baseline lineal. XGBoost mejora al baseline en accuracy (0.7516 vs. 0.72) y F1 (0.6285 vs. 0.6164), lo que sugiere que las interacciones no lineales entre *tenure*, *Contract* y *MonthlyCharges* aportan valor incremental para identificar correctamente los casos positivos, aunque no mejoran la discriminación global medida por AUC.

Random Forest alcanza el AUC más alto (0.843) pero fue descartado como modelo de producción por dos razones prácticas: (1) su F1 y accuracy no superaron a XGBoost en las métricas registradas, y (2) XGBoost ofrece mejor integración nativa con SHAP, facilitando la explicabilidad local por cliente.

El análisis SHAP valida cuantitativamente la intuición de negocio: un cliente con contrato mes a mes, baja antigüedad y cargo mensual elevado concentra los tres factores de riesgo más importantes. Esto permite que las

recomendaciones del LLM sean específicas y accionables, no genéricas.

El componente LLM agrega valor al traducir el score numérico en un párrafo orientado a la acción. La estrategia zero-shot con *temperature=0.4* produce respuestas coherentes y consistentes entre ejecuciones, aunque la calidad depende de que el prompt incluya los atributos correctos. Los nueve atributos seleccionados cubren los factores identificados en el EDA y son suficientes para que el modelo genere explicaciones pertinentes sin sobrecargar el contexto.

6.2. Limitaciones

- El dataset corresponde a una empresa ficticia de IBM y puede no generalizarse a otras regiones o sectores de telecomunicaciones con estructuras de contrato o tarifas distintas.
- No se realizó calibración de probabilidades (e.g., isotonic regression o Platt scaling), por lo que los scores pueden estar sesgados; esto afecta la interpretación directa de la probabilidad predicha.
- La explicación del LLM depende de la calidad del prompt y del modelo subyacente. Sin validación humana sistemática, no debe usarse directamente en producción para decisiones de alto impacto.
- El split hold-out no sustituye a la validación cruzada para estimar la varianza del desempeño; los resultados podrían variar con distintas semillas.
- Random Forest no fue completamente caracterizado (F1 y accuracy no registrados), lo que impide una comparación exhaustiva de los tres modelos.

6.3. Trabajo futuro

- Implementar validación cruzada estratificada ($k = 5$) y búsqueda sistemática de hiperparámetros con Optuna para reducir la varianza de las estimaciones de desempeño.
- Calibrar las probabilidades del modelo con isotonic regression para mejorar la confiabilidad de los scores de riesgo.
- Integrar un componente RAG que recupere políticas de retención relevantes de una base de conocimiento interna y las incluya en el prompt del LLM, mejorando la especificidad de las recomendaciones.
- Registrar F1 y accuracy de Random Forest para completar la comparación de modelos y decidir formalmente si XGBoost es la mejor opción de producción.
- Evaluar la calidad de las explicaciones del LLM mediante un protocolo de anotación humana que mida coherencia, precisión y utilidad percibida.

7. Conclusiones

El proyecto entregó un pipeline reproducible para la detección de churn en telecomunicaciones que combina un modelo XGBoost con explicabilidad SHAP y generación automática de reportes en español vía Groq. La comparación de tres modelos (Regresión Logística, Random Forest y XGBoost) mostró que todos convergen en un rango AUC de 0.839–0.843, confirmando que la mayor parte de la información discriminante está presente en las variables clave identificadas en el EDA: Contract, tenure y MonthlyCharges.

XGBoost fue seleccionado como modelo de producción por su mayor F1 (0.6285) y accuracy (0.7516), métricas más relevantes para el caso de uso de retención donde importa identificar correctamente a los clientes en riesgo. La integración con SHAP permite explicar cada predicción a nivel individual, y el componente LLM traduce esas explicaciones en recomendaciones accionables en español accesibles para usuarios no técnicos. La interfaz en Streamlit permite validar casos individuales de forma interactiva, completando el ciclo desde los datos hasta la decisión de negocio.

Demo de la Aplicación

La interfaz desarrollada en Streamlit permite evaluar clientes de forma interactiva sin conocimientos técnicos. El usuario ingresa los atributos del cliente en la barra lateral y obtiene inmediatamente la probabilidad de churn, el nivel de riesgo y una explicación en lenguaje natural generada por Groq (Figura 8).



Figura 8: Vista del formulario y resultado de la app con predicción.

Contribuciones del equipo

Integrante	Contribución principal
Kadiha Muhamad Orta	EDA, limpieza de datos, visualizaciones
Laura Restrepo Berrio	Modelado (LR + RF + XGBoost), SHAP, métricas
Johan Rico Nivia	LLM (Groq), app Streamlit, informe final

Distribución de responsabilidades.

Repositorio y Demo

- **GitHub:** <https://github.com/Johan-Rico/Churn-Prediction-LLM>
- **Videos:** <https://drive.google.com/drive/u/1/folders/1fGfWmflEh9fus8RI56SUsAoqlHVsjL0a>
- **Instalación:** `pip install -r requirements.txt`
- **Notebooks:** 01_eda.ipynb, 02_modeling.ipynb, 03_llm_evaluation.ipynb
- **App:** `streamlit run app/main.py`

Apéndice: Hiperparámetros

Cuadro 4: Búsqueda y parámetros finales para el modelo XGBoost

Aspecto	Valor / Nota
Método	GridSearchCV (roc_auc, cv=3)
Parámetros	max_depth: [3, 4, 5], lr: [0.01, 0.05, 0.1], n_est: [200, 300]
Métrica	AUC-ROC
Mejores params.	lr=0.05, depth=3, n=200
AUC test set	0.8393 (Tabla 3)

8. Ejemplos adicionales de outputs del LLM

En esta sección se muestran 3 ejemplos representativos de explicaciones automáticas generadas por Groq para distintos perfiles de cliente (alto, medio y bajo riesgo). Cada ejemplo incluye: (1) entrada usada por el modelo, (2) probabilidad predicha por el clasificador y (3) explicación textual generada por el LLM.

Ejemplo A — Alto riesgo

```
1 cliente = {"tenure": 2, "Contract":  
2           "Month-to-month",  
3           "MonthlyCharges": 89.5,  
           "InternetService": "Fiber optic"}
```

```
4 prob_predicha = 0.82
5
6 Explicacion (Groq):
7 "Este cliente tiene una probabilidad de
8 abandono del 82%.
9 Los factores clave son contrato mensual, baja
10 antigüedad
11 (2 meses) y cargos mensuales elevados.
12 Recomendamos
13 ofrecer un descuento por contrato anual o un
14 plan con
15 menor cargo mensual."
```

Ejemplo B — Riesgo medio

```
1 cliente = {"tenure": 18, "Contract": "One
2 year",
3           "MonthlyCharges": 70.0,
4           "InternetService": "DSL"}
5 prob_predicha = 0.45
6
7 Explicacion (Groq):
8 "Probabilidad de abandono moderada (45%). El
9 contrato
10 anual reduce el riesgo, pero cargos
11 relativamente altos
12 y servicios adicionales incrementan la
13 probabilidad.
14 Sugerir revision de factura o beneficios
15 adicionales."
```

Ejemplo C — Bajo riesgo

```
1 cliente = {"tenure": 60, "Contract": "Two
2 year",
3           "MonthlyCharges": 40.0,
4           "InternetService": "DSL"}
5 prob_predicha = 0.06
6
7 Explicacion (Groq):
8 "Bajo riesgo de abandono (6%). Cliente con
9 larga
10 antigüedad y contrato a dos años.
11 Recomendaciones:
12 mantener incentivos actuales y monitorear
13 variaciones
14 de factura."
```

Referencias

- [1] IBM. (2018). *Telco Customer Churn*. Kaggle. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- [2] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD 2016. <https://arxiv.org/abs/1603.02754>
- [3] Lundberg, S., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. NeurIPS 2017. <https://arxiv.org/abs/1705.07874>
- [4] Groq. (2024). *Groq API Documentation*. <https://console.groq.com/docs>

Interpretación y uso: Estos ejemplos deben acompañarse de capturas reales tomadas desde la app Streamlit (`streamlit run app/main.py`) o desde el notebook `03_llm_evaluation.ipynb`. Se recomienda una imagen donde se muestre simultáneamente los atributos del cliente, la probabilidad numérica y el párrafo generado por el LLM.